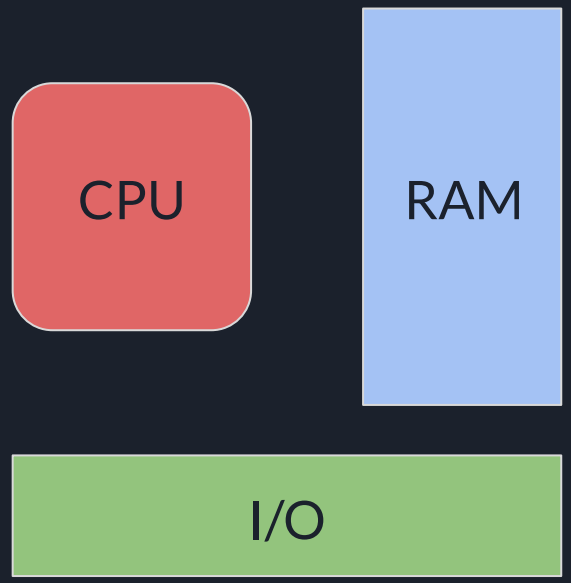
Multithreading vs Multiprocessing vs Async

IADES – Programación II

Repaso .... Arquitectura de una computadora

Arquitectura Von Neumann



Repaso .... Arquitectura de una computadora

interacción del programa con la computadora

```
# Almacenamiento en memoria
x = 1

# Uso de  CPU
x += 3
```

```
# I/O (Escritura archivo)
with open('res.txt', 'w') as fp:
fp.write(f"The value is {x}")

# I/O (salida pantalla)
print(x)
```

Tiempos de acceso

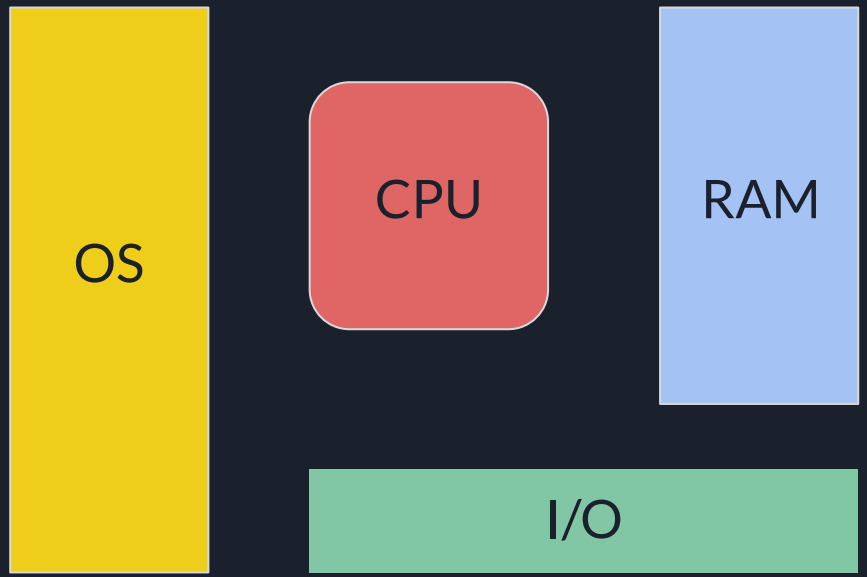
Relativos a un 1 segundo

CPU Cycle	1 second
RAM Access	4 minutes
Generic SSD Access	1.5–4 days
Hard Drive access	1-9 months
Network Request SF -> NYC	5 years
Network Request SF -> Hong Kong	11 years

El Sistema Operativo

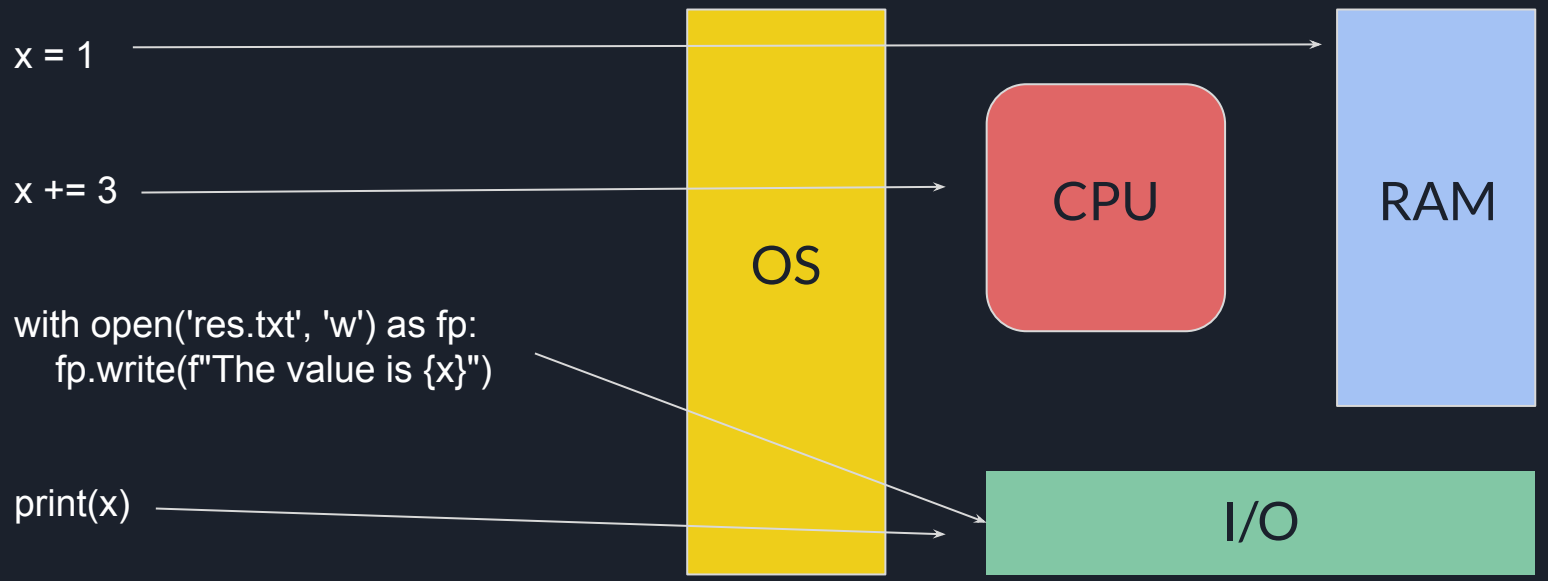
El S.O. está entre nuestro programa y el hardware

```
x = 1

x += 3

with open('res.txt', 'w') as fp:
    fp.write(f"The value is {x}")

print(x)
```



El Sistema Operativo

El S.O. está entre nuestro programa y el hardware



S.O. – Procesos

Supongamos…… proceso.py

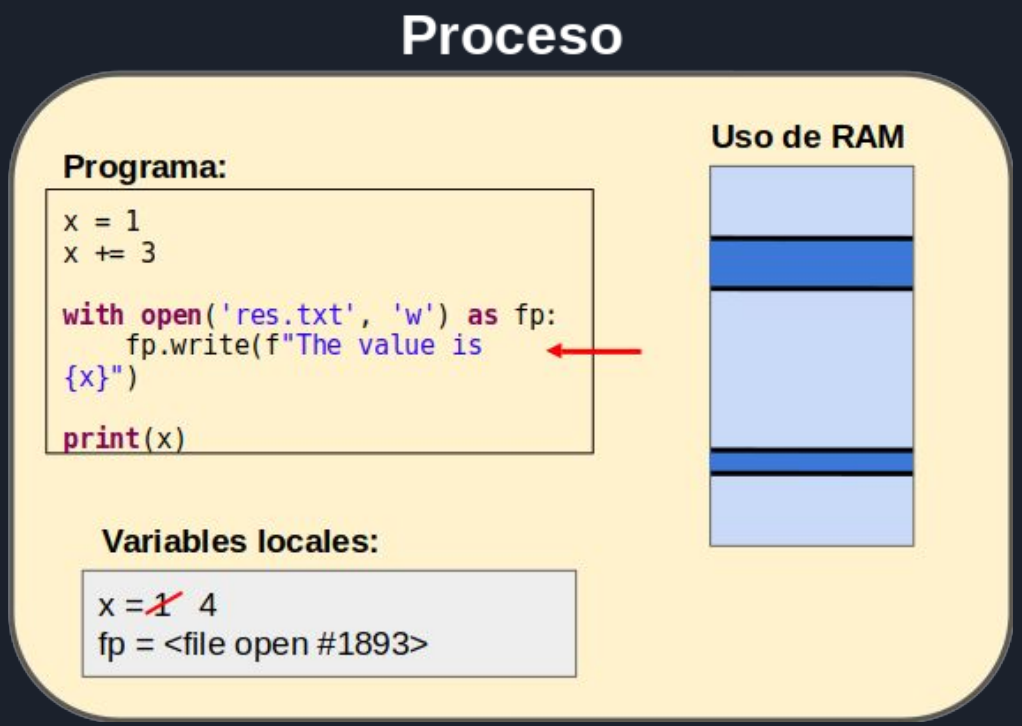
```
# Datos
x = 1
# Cálculo
x += 3
# I/O  Entrada/Salida
with open('res.txt', 'w') as fp:
   fp.write(f"The value is {x}")

print(x)
```

S.O. – Procesos

El SO envuelve nuestro programa en una estructura especial llamada proceso.

Un proceso mantiene el estado de ejecución de una "instancia" determinada del programa en ejecución.

Es por eso que podemos tener varias instancias del mismo programa ejecutándose al mismo tiempo.



S.O. – Procesos

Podemos correr varias instancias de un mismo programa al mismo tiempo

```
pablo  ~/Documents/IADES/2024/programacion_III/multi  1  python3 sleep_n_seconds.py 60 &
[1] 18373
pablo  ~/Documents/IADES/2024/programacion_III/multi  python3 sleep_n_seconds.py 70 &
[2] 18389
pablo  ~/Documents/IADES/2024/programacion_III/multi  python3 sleep_n_seconds.py 80 &
[3] 18406
pablo  ~/Documents/IADES/2024/programacion_III/multi
```

```
pablo  ~/Documents/IADES/2024/programacion_III/multi  ps -eo pid,ppid,%mem,%cpu,args | { head -1 ; grep "sleep_n_seconds" ; }
   PID    PPID %MEM %CPU COMMAND
 18373    5402  0.0  0.0 python3 sleep_n_seconds.py 60
 18389    5402  0.0  0.0 python3 sleep_n_seconds.py 70
 18406    5402  0.0  0.2 python3 sleep_n_seconds.py 80
```

Distintos PID y el mismo PPID

Volviendo al tema de la charla.....

Cuál uso? Donde ?

Cuándo? Por qué?

Empecemos por acá.....

Concurrencia y Paralelismo

Que son?

Concurrencia

“Hacer muchas cosas juntas”

Paralelismo

“Hacer muchas cosas al mismo tiempo”

Cómo sería?

Concurrencia

“ Jose mira fotos y dibuja ”

“ Jose mira fotos y Pedro dibuja”

Paralelismo

“ Jose mira fotos y Pedro dibuja”

Entonces.....

Paralelismo: una forma de lograr

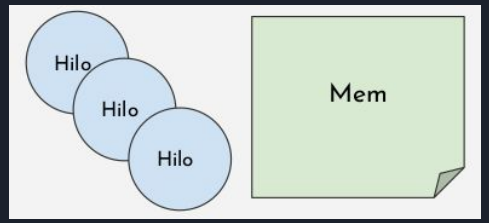
Concurrencia

Pero no la única

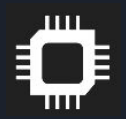
Y en computación?

Supongamos que tenemos una computadora que está corriendo dos procesos al mismo tiempo con las siguientes características…… el microprocesador ejecuta los procesos……

Proceso 1



Proceso 2

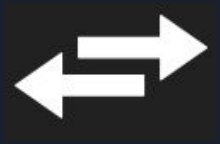


1 core

1 proceso/hilo corriendo en cada instante

Concurrencia (“un poquito cada uno”) pero **no** Paralelismo

Paralelismo implica N cores



Context switches

Cambiar de hilo/proceso tiene su costo de CPU

Todo un mundo de nuevos problemas y soluciones (ej: race conditions, locks, etc)

Esperas de I/O

Esperas de operaciones de disco, red, etc

Cuánto? depende del programa

*Si un hilo/proceso está **esperando**, tiene sentido “darle” tiempo de cpu?*

Atención

Es fácil compartir datos entre hilos de un mismo proceso.

Es más complicado compartir datos en multiprocessing

Programas sin esperas de I/O

Paralelismo ayuda a reducir tiempos

Concurrencia sin Paralelismo **no** los reduce!

1 core? Nada nos va a ayudar 🙁

Programas con esperas de I/O - I

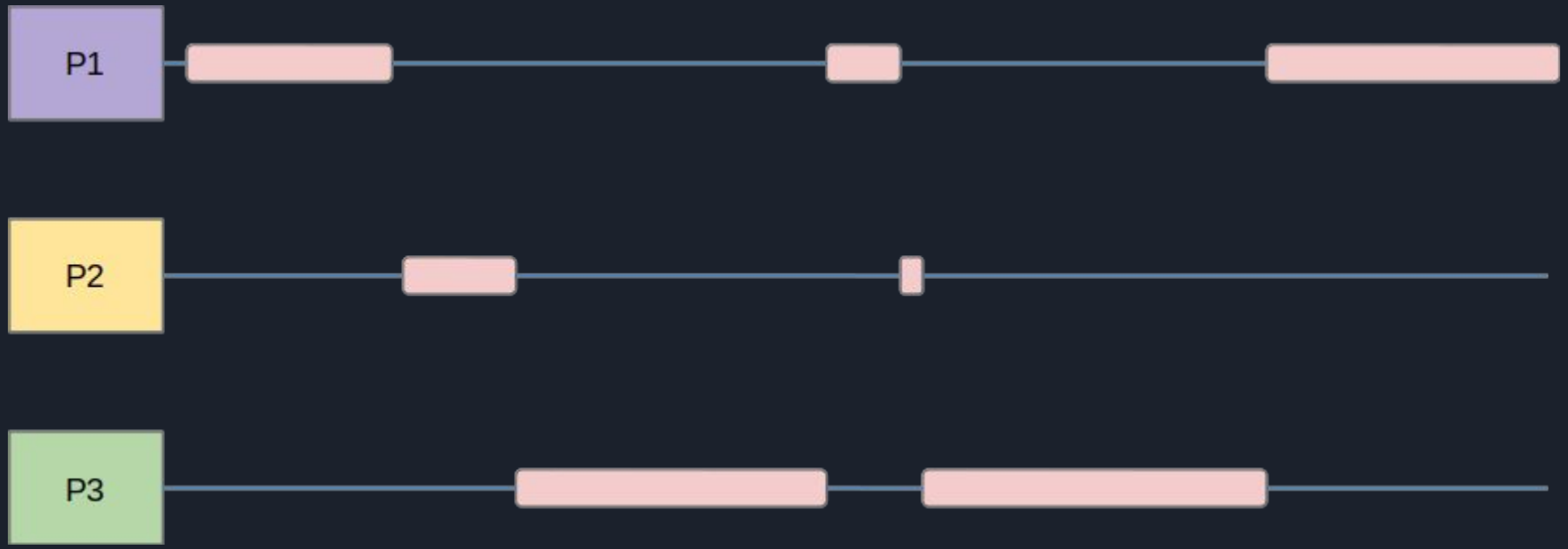
Concurrencia sin Paralelismo **igual ayuda** a acelerar

Programas con esperas de I/O - II

Lo más óptimo es combinar ambas cosas!!!

Paralelismo con varios cores, concurrencia en cada core

Concurrencia – 1 solo CPU
No hay procesos corriendo en paralelo



Time Slicing

El sistema operativo tiene un “scheduler” que se encarga de administrar el tiempo de CPU de cada uno de los procesos que están corriendo.

Volviendo a Python!!!

Multi threading

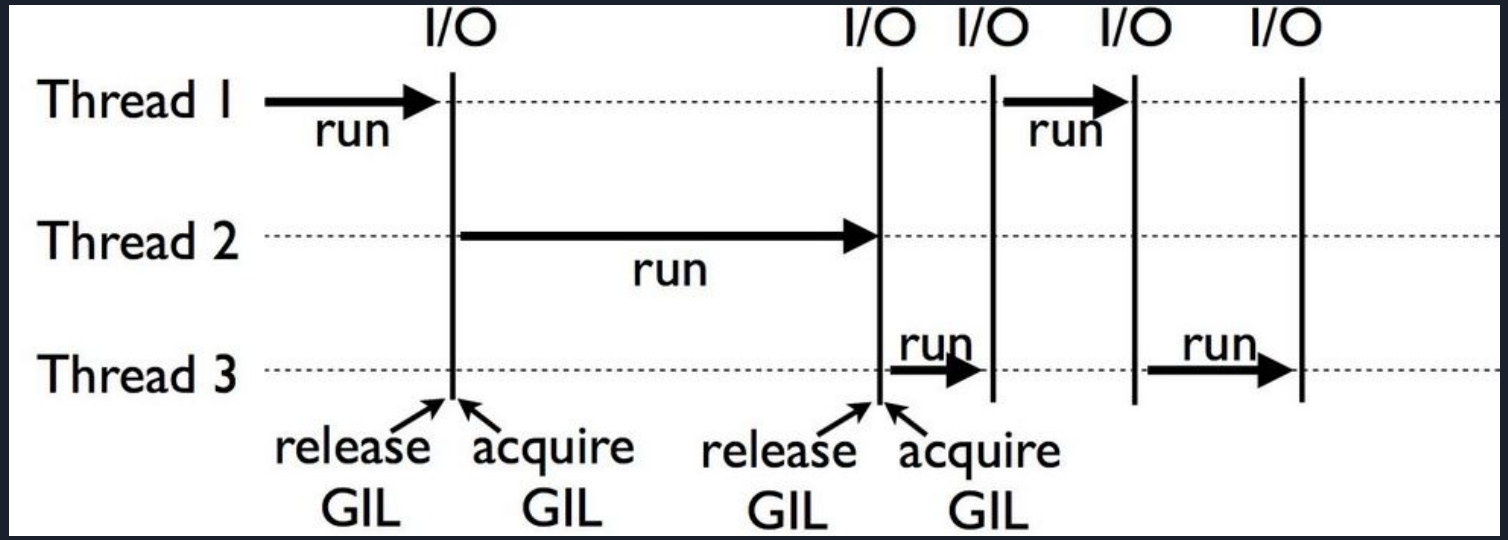
Simple! Módulo **threading** permite lanzar hilos y controlarlos.

Paralelismo?
No en CPython!* (el oficial)
Sí en otros pythons (N cores) Jython

(*) por la famosa GIL

GIL
Global Interpreter Lock

Por más que tengamos N cores nuestro programa se va a ejecutar en un solo core



Multi threading

Los hilos comparten memoria 😊

Ejemplo: Creo una lista en un hilo y agrego items desde otro hilo sin hacer nada.

Multi threading

En casos de muchas tareas (hilos) o esperas de I/O muy largas, el context switch hace que no sea eficiente.

Veamos un ejemplo.....

Multi processing

Simple! El módulo **multiprocessing** nos permite lanzar procesos y controlarlos.

Paralelismo? Si hay N cores sí! Puentea al GIL

Multi processing

Cada uno con su memoria

Mucha memoria repetida
(*python, libs, etc*)

Si quiero compartir datos?
Comunicación entre procesos, no tan trivial, pero Python ayuda

Multi processing

En casos de muchas tareas (procesos) o esperas de I/O muy largas, el context switch hace que no sea eficiente.

Veamos un ejemplo.....

y async ???

Async

1 proceso, 1 hilo, N corrutinas

Corrutinas: funciones que se pueden pausar para dejar que otras corran

Un loop va haciendo correr un poco cada una, respetando las pausas

Async

1 proceso → Memoria compartida, como en multithreading

Async

1 hilo → Solo Concurrencia, **no** Paralelismo

Pero **nosotros** decimos cuándo se hace el context switch y hasta cuándo

Ej: “pausá esta corrutina hasta que termine de escribirse tal dato al disco”

Async

Más eficiente, sobre todo en mucha carga de tareas o I/O

Más seguro/controlable/reproducible

Un poco más complicado
A la vez, más natural a la hora de controlar context switches

No todas las libs lo soportan

Veamos un ejemplo.....

Conclusiones.....

Concurrencia

vs

Paralelismo

vs

Async

Programa sin esperas de I/O?

Multiprocessing y N cores

Ninguna otra cosa va a ayudar

Programa con esperas de I/O? - I

Si la **performance** es muy importante:

Nada le gana a Async+Multiprocessing, pagando el costo de la complejidad

Programa con esperas de I/O? – II

Si el **control** de los context switches o **reproducibilidad** es importante:

Async suele hacerlo más natural, una vez que lo aprendiste a usar

Programa con esperas de I/O? - III

Si no, Multithreading si hay mucha/compleja **data compartida**, o Multiprocessing en caso contrario